Limited Time Offer: **Up to 25% off select mock interviews** 🎉

Patterns

Handling Large Blobs

Learn about how to handle large blobs in your system design interview.

Handling Large Blobs

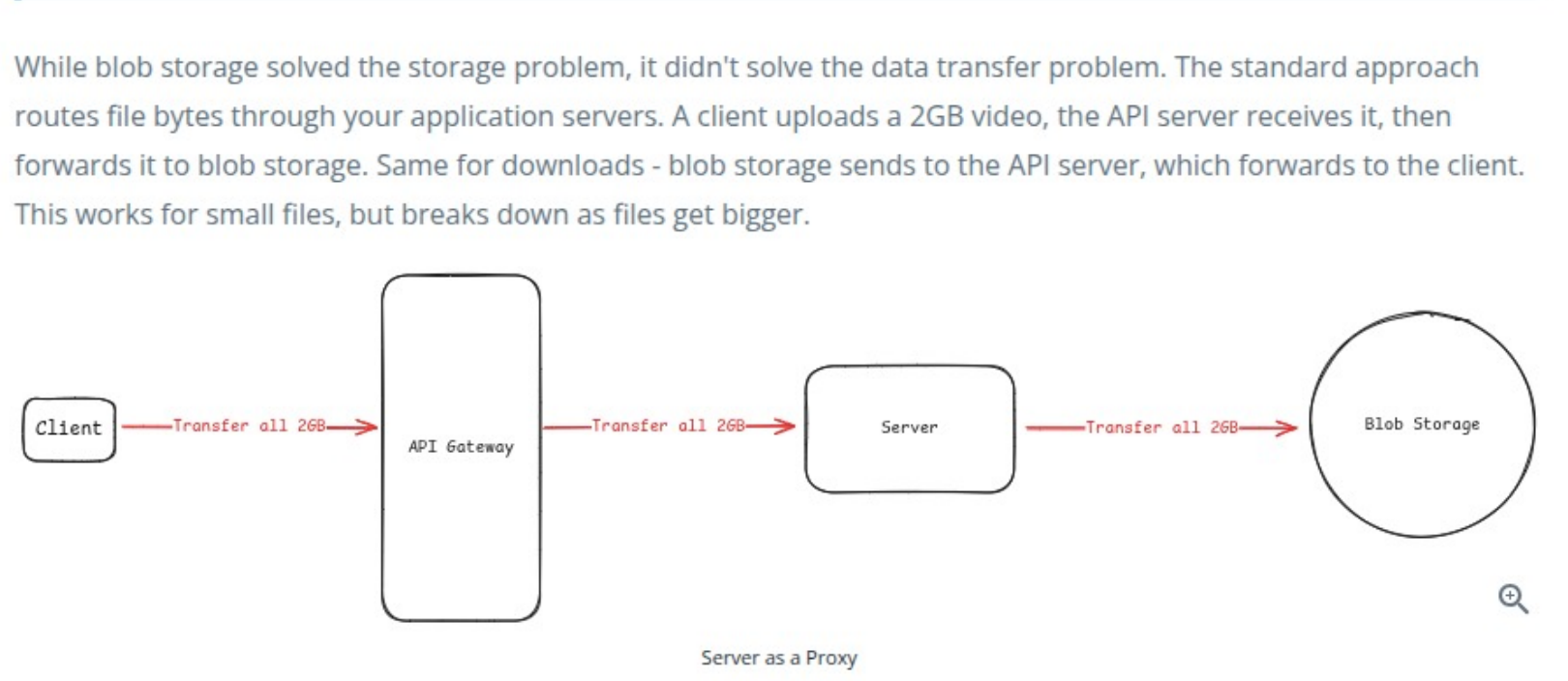
📁 Large files like videos, images, and documents need special handling in distributed systems. Instead of shoving gigabytes through your servers, this pattern uses presigned URLs to let clients upload directly to blob storage and download from CDNs. You also get resumable uploads, parallel transfers, and progress tracking - the stuff that separates real systems from toy projects.

The Problem

If you've been studying for system design interviews, you already know that large files belong in blob storage like S3, not in databases. This separation lets storage scale independently from compute and keeps database performance snappy.

> **Why blob storage?**
> Databases are great at structured data with complex queries but terrible with large binary objects. A 100MB file stored as a BLOB kills query performance, backup times, and replication. Object stores like S3 are built for this: unlimited capacity, 99.999999999% (11 nines) durability, and per-object pricing. As a general rule of thumb, if it's over 10MB and doesn't need SQL queries, it should probably be in blob storage.

While blob storage solved the storage problem, it didn't solve the data transfer problem. The standard approach routes file bytes through your application servers. A client uploads a 2GB video, the API server receives it, then forwards it to blob storage. Same for downloads - blob storage sends to the API server, which forwards to the client. This works for small files, but breaks down as files get bigger.



Server as a Proxy

We're forcing application servers to be dumb pipes. They add no value to the transfer, just latency and cost. Cloud providers already have global infrastructure, resume capability, and massive bandwidth. Yet we insert our limited application servers as middlemen, creating a bottleneck where none needs to exist.

So (spoiler alert) we switch from uploading through our servers to uploading directly from the client to blob storage. Meanwhile, downloads come from global CDNs like CloudFront. Your servers never touch the actual bytes. This solves the proxy bottleneck but on its own it's not enough, because even more problems pop up.

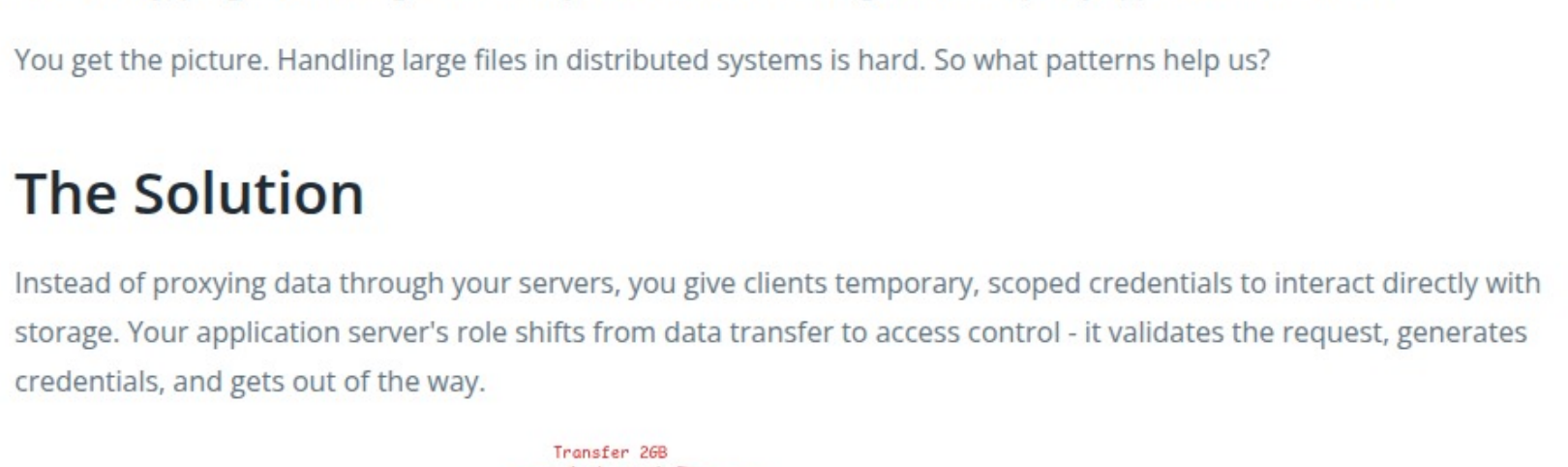
That 25GB upload failing at 99% means starting over from scratch. You lose visibility into upload progress since it bypasses your servers entirely. Your database shows "pending" while S3 might already have the complete file - or maybe it doesn't, and you can't tell without checking.

The seemingly simple act of moving bytes around has become a distributed systems problem with eventual consistency, progress tracking, and state synchronization challenges that the proxy approach never faced.

You get the picture. Handling large files in distributed systems is hard. So what patterns help us?

The Solution

Instead of proxying data through your servers, you give clients temporary, scoped credentials to interact directly with storage. Your application server's role shifts from data transfer to access control - it validates the request, generates credentials, and gets out of the way.



Serverless Data Access

Whether you're using AWS S3, Google Cloud Storage, or Azure Blob Storage, they all support temporary URLs that grant time-limited upload or download permissions. CDNs can validate signed tokens before serving content too.

Think of your application server as a ticket booth at a concert venue. When someone shows up with a valid ticket, you don't personally escort them to their seat. You verify their identity, hand them a wristband, and let them find their own way. Same thing here. We validate authorization once, then let clients interact with storage directly.

Simple Direct Upload

When a client wants to upload a file, instead of receiving the entire file, your server just receives a request for upload permission. You validate the user, check quotas, then generate a temporary upload URL. This URL encodes permission to upload one specific file to one specific location for a limited time - typically 15 minutes to 1 hour. This is called a **presigned URL**.

Generating a presigned URL happens entirely in your application's memory - no network call to blob storage needed. Your server uses its cloud credentials to create a signature that the storage service can verify later. Here's what a presigned URL actually looks like:

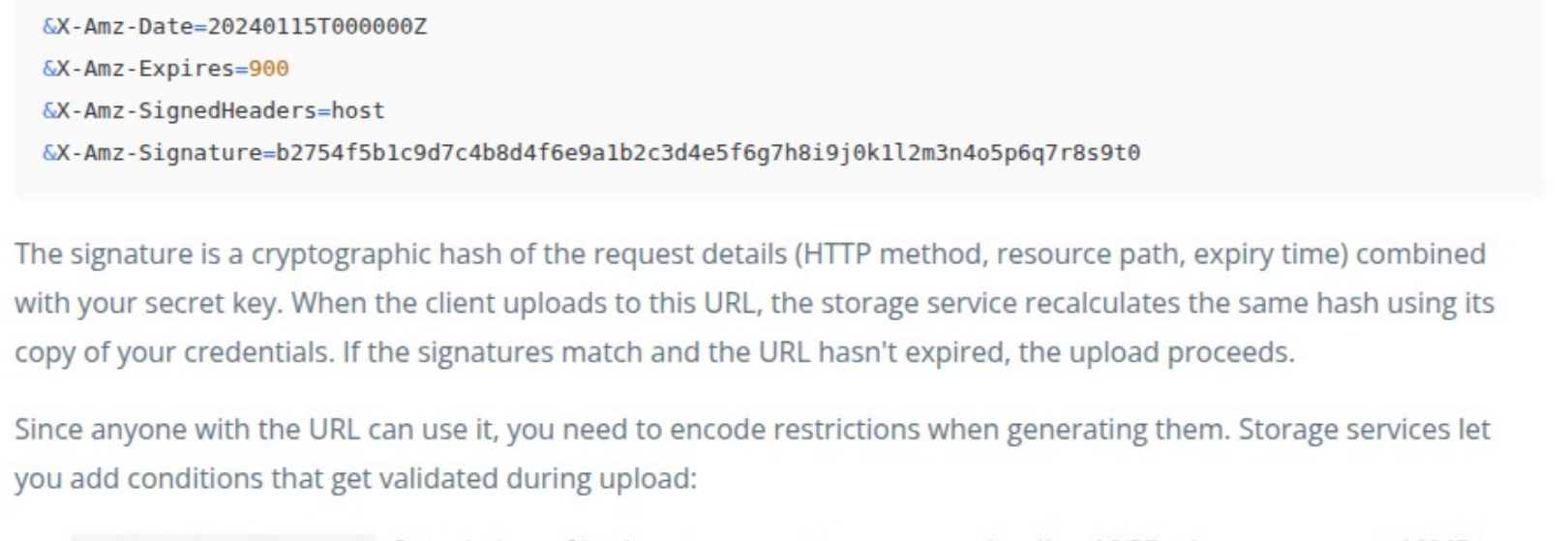
```
https://mybucket.s3.amazonaws.com/uploads/user123/video.mp4
?X-Amz-Algorithm=AWS4-HMAC-SHA256
&X-Amz-Credential=AKIAIOSFODNN7EXAMPLE%2F20240115%2Fus-east-1%2Fs3%2Faws4_request
&X-Amz-Date=20240115T000000Z
&X-Amz-Expires=900
&X-Amz-SignedHeaders=host
&X-Amz-Signature=b2754f5b1c9d7c4b8d4f6e9a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0
```

The signature is a cryptographic hash of the request details (HTTP method, resource path, expiry time) combined with your secret key. When the client uploads to this URL, the storage service recalculates the same hash using its copy of your credentials. If the signatures match and the URL hasn't expired, the upload proceeds.

Since anyone with the URL can use it, you need to encode restrictions when generating them. Storage services let you add conditions that get validated during upload:

- `content-length-range` : Set min/max file sizes to prevent someone uploading 10GB when you expect 10MB
- `content-type` : Ensure that profile picture endpoint only accepts images, not videos

These restrictions are baked into the signature. A URL generated for a 5MB image upload will be rejected if someone tries to upload a 500MB video.

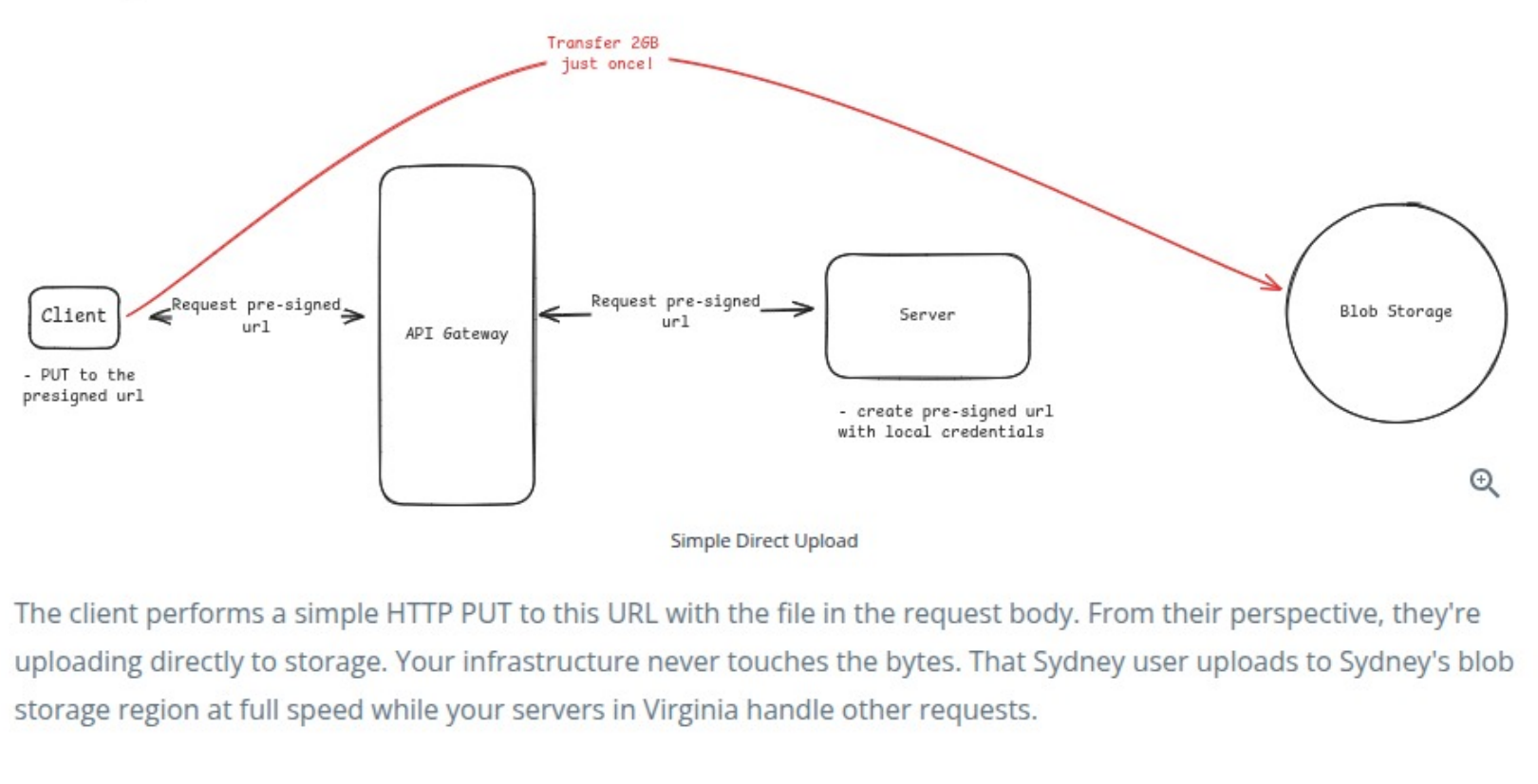


Simple Direct Upload

The client performs a simple HTTP PUT to this URL with the file in the request body. From their perspective, they're uploading directly to storage. Your infrastructure never touches the bytes. That Sydney user uploads to Sydney's blob storage region at full speed while your servers in Virginia handle other requests.

Simple Direct Download

Downloads work the same way, either directly from blob storage or through a CDN. You generate signed URLs that grant temporary read access to specific files. Direct blob storage access is simpler and cheaper for infrequent downloads. CDN distribution costs more but gives better performance for frequently accessed files through geographic distribution and caching.


CDN close to the users location
CDN
Fetch from blob storage only on cache miss
Download
Client
API Gateway
Server
Blob Storage

Simple Direct Download

For CDN delivery, you're not signing blob storage URLs. Instead, you're creating signed URLs or signed cookies that your CDN validates. Here's how CloudFront handles it:

```
https://d123456.cloudfront.net/videos/lecture.mp4
?Expires=1705305600
&Signature=j1k2l3m4n5o6p7q8r9s0t1u2v3w4x5y6z7a8b9c0d1e2f3g4h5i6j7k8l9m0
&Key-Pair-Id=APKAIOSFODNN7EXAMPLE
```

Both approaches use signatures, but the key difference is **who validates them**:

Blob storage signatures (like S3 presigned URLs) are validated by the storage service using your cloud credentials. The storage service has your secret key and can verify that you generated the signature.

CDN signatures (like CloudFront signed URLs) are validated by the CDN edge servers using public/private key cryptography. You hold the private key and sign the URL. The CDN has the corresponding public key and validates signatures at edge locations worldwide - no need to call back to your servers or the origin storage service.

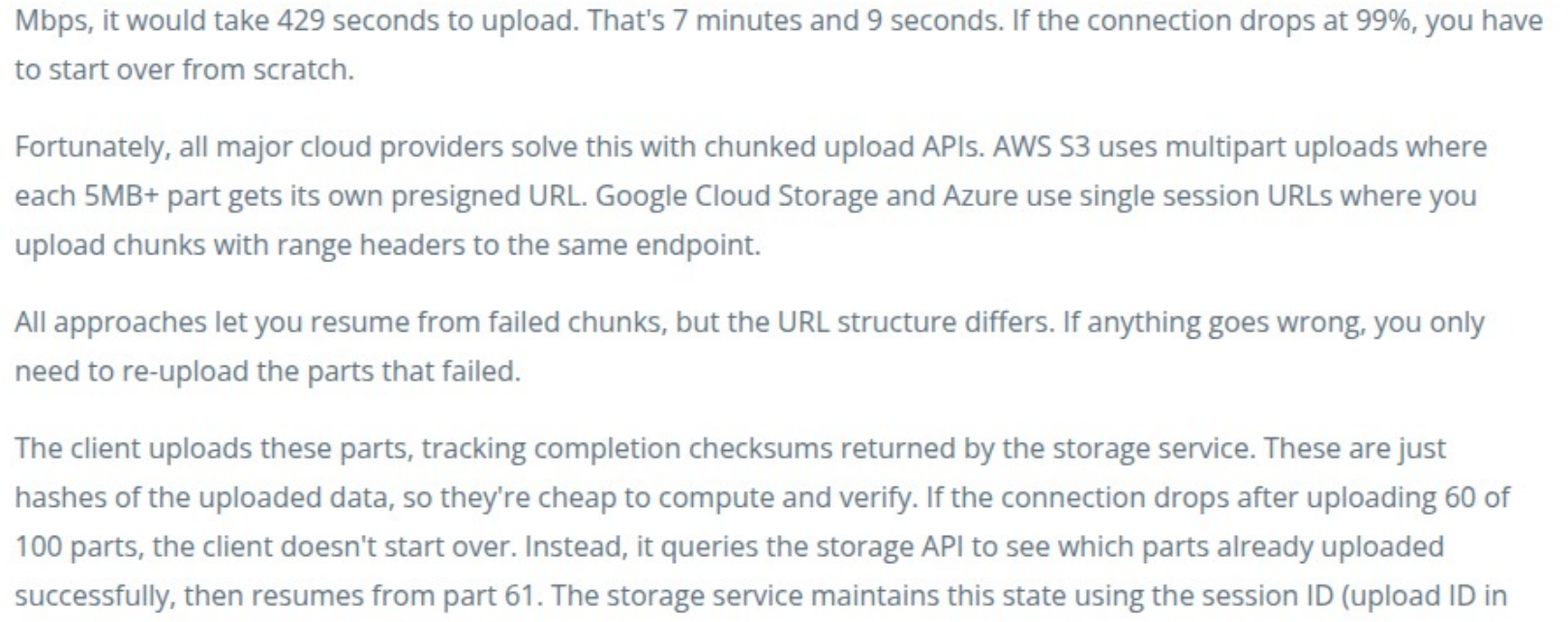
Resumable Uploads for Large Files

Consider what happens when uploading really large files. Say a 5 GB video. With a typical network connection of 100 Mbps, it would take 429 seconds to upload. That's 7 minutes and 9 seconds. If the connection drops at 99%, you have to start over from scratch.

Fortunately, all major cloud providers solve this with chunked upload APIs. AWS S3 uses multipart uploads where each 5MB+ part gets its own presigned URL. Google Cloud Storage and Azure use single session URLs where you upload chunks with range headers to the same endpoint.

All approaches let you resume from failed chunks, but the URL structure differs. If anything goes wrong, you only need to re-upload the parts that failed.

The client uploads these parts, tracking completion checksums returned by the storage service. These are just hashes of the uploaded data, so they're cheap to compute and verify. If the connection drops after uploading 60 of 100 parts, the client doesn't start over. Instead, it queries the storage API to see which parts already uploaded successfully, then resumes from part 61. The storage service maintains this state using the session ID (upload ID in S3, resumable upload URL in GCS, etc.).



Resumable Uploads

> Progress tracking comes naturally with chunked uploads. As each part completes, you know the percentage done. Simple client-side math gives you a nice progress bar to show users.

Finally, after all parts upload, the client must call a completion endpoint with the list of part numbers and their checksums. The storage service then assembles these parts into the final object. Until this completion succeeds, you have parts in storage but no accessible file. Incomplete multipart uploads cost money, so lifecycle rules should clean them up after 24-48 hours.

> Once multipart upload completes, the "chunks" no longer exist from the perspective of the storage service. They're all combined into a single object and it's as if you uploaded the entire file in one go.

State Synchronization Challenges

Moving your servers out of the critical path solves the bottleneck, but introduces a new class of problems around distributed state management.

The common pattern is to store file metadata in your database while the actual file lives in blob storage. This gives you the best of both worlds - fast queries on metadata with unlimited storage for the files themselves.

Consider a file storage system like Dropbox. Your metadata table might look like:

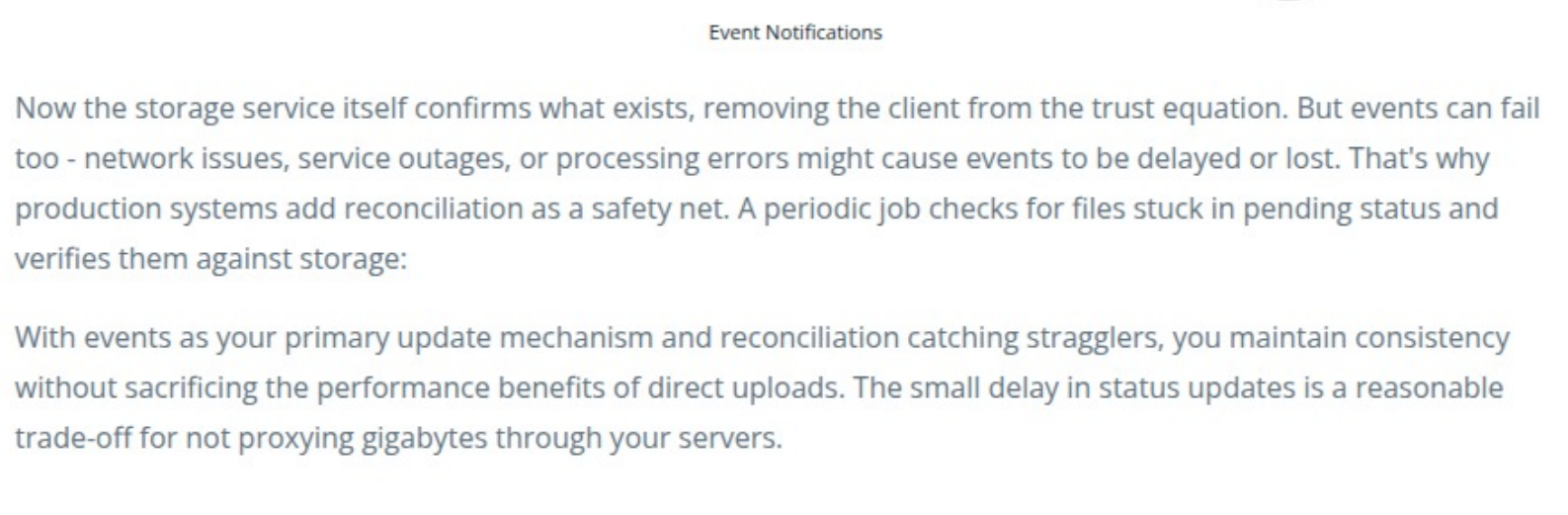
```sql
CREATE TABLE files (
    id              UUID PRIMARY KEY,
    user_id         UUID NOT NULL,
    filename        VARCHAR(255),
    size_bytes      BIGINT,
    content_type    VARCHAR(100),
    storage_key     VARCHAR(500),  -- s3://bucket/user123/files/abc-123.pdf
    status          VARCHAR(50),   -- 'pending', 'uploading', 'completed', 'failed'
    created_at      TIMESTAMP,
    updated_at      TIMESTAMP
);
```

The `status` column tells you whether a file is ready to download or still uploading. But with direct uploads, keeping this in sync gets tricky - your database and blob storage are separate systems that update at different times.

The simplest approach is trusting the client. After uploading to blob storage, the client calls your API saying "upload complete!" and you update the database status. But this creates several problems:

1. **Race conditions**: The database might show 'completed' before the file actually exists in storage
2. **Orphaned files**: The client might crash after uploading but before notifying you, leaving files in storage with no database record
3. **Malicious clients**: Users could mark uploads as complete without actually uploading anything
4. **Network failures**: The completion notification might fail to reach your servers

Most blob storage services solve this with event notifications. When S3 receives a file, it publishes events through messaging services (like SNS/SQS) with details about what was uploaded. The event includes the object key - the same `storage_key` you stored in your database when generating the presigned URL. This lets you find the exact row to update:


Upload in 5MB chunks
Chunk 4
Chunk 5
Chunk 6
Chunk 3
Chunk 2
Chunk 1
- stitch the chunks back together
Client
- responsible for chunking (with cloud provider SDK)
API Gateway
Server
Blob Storage
Write Metadata
Event notification to update completion status
Primary DB

Event Notifications

Now the storage service itself confirms what exists, removing the client from the trust equation. But events can fail too - network issues, service outages, or processing errors might cause events to be delayed or lost. That's why production systems add reconciliation as a safety net. A periodic job checks for files stuck in pending status and verifies them against storage:

With events as your primary update mechanism and reconciliation catching stragglers, you maintain consistency without sacrificing the performance benefits of direct uploads. The small delay in status updates is a reasonable trade-off for not proxying gigabytes through your servers.

Cloud Provider Terminology

Each provider has their own names for the same concepts.

> ⚠ The exact names of the SDK functions and parameters vary by provider and are not something you typically need to know for an interview so long as you understand the main idea. That said, I often get asked, "What is the equivalent in X provider?" so figured there was value in including this table.

Feature	AWS	Google Cloud	Azure
Temporary Upload URLs	Presigned URLs (PUT or POST)	Signed URLs (resumable or simple)	SAS tokens (Shared Access Signature)
Multipart Uploads	Multipart Upload API (5MB-5GB parts)	Resumable Uploads (flexible chunk sizes)	Block Blobs (multipart equivalent, 4MB-100MB blocks)
Event Notifications	S3 Event Notifications (to Lambda/SQS/SNS)	Cloud Storage Pub/Sub (to Cloud Functions)	Event Grid (blob storage events)
CDN with Signed URLs	CloudFront (signed URLs/cookies)	Cloud CDN (signed URLs)	Azure CDN (SAS tokens)
Cleanup Policies	Lifecycle Rules	Lifecycle Management	Lifecycle Management Policies

When to Use in Interviews

The rule is simple - no need to overthink it. If you're moving files larger than 100MB through your API, you should immediately think of this pattern. The exact threshold depends on your infrastructure, but 100MB is where the pain becomes real.

Common interview scenarios

YouTube/Video Platforms - Video uploads are the perfect use case. A user uploads a 2GB 4K video file. The web app generates a presigned S3 URL, the client uploads directly to S3 (often using multipart for resumability), and S3 events trigger transcoding workflows. Downloads work similarly - CloudFront serves video segments with signed URLs, enabling adaptive bitrate streaming without your servers ever touching the video bytes.

Instagram/Photo Sharing - Photo uploads bypass the API servers entirely. The mobile app gets presigned URLs for original images (which can be 50MB+ from modern cameras), uploads directly to S3, then S3 events trigger async workers to generate thumbnails, apply filters, and extract metadata. The feed serves images through CloudFront with signed URLs to prevent hotlinking.

Dropbox/File Sync - File uploads and downloads are pure serverless data access. When you drag a 500MB video into Dropbox, it gets presigned URLs for chunked uploads, uploads directly to blob storage, then triggers sync workflows. File shares generate time-limited signed URLs that work without the recipient having a Dropbox account.

WhatsApp/Chat Applications - Media sharing (photos, videos, documents) uses presigned URLs. When you send a video in WhatsApp, it uploads directly to storage, then the chat system only passes around the file reference. Recipients get signed download URLs that expire after a period, ensuring media can't be accessed indefinitely.

When NOT to use it in an interview

This pattern adds complexity that's only worth it when you're dealing with actual scale or size problems.

Small files don't need it. Anything under 10MB - JSON payloads, form submissions, small images - should use normal API endpoints. The two-step dance adds latency and complexity for no real benefit. Your servers can handle thousands of small requests without the overhead of generating presigned URLs.

Synchronous validation requirements. If you need to reject invalid data before accepting the upload, you need to proxy it. For example, a CSV import where you must validate headers and data types before confirming the upload. You can still process data asynchronously after upload, but real-time validation during upload requires seeing the bytes as they flow.

Compliance and data inspection. Some regulatory frameworks require that data passes through certified systems or gets scanned before storage. Financial services scanning for credit card numbers, healthcare systems enforcing HIPAA requirements, or any system that must inspect data for compliance reasons needs the traditional proxy approach. In these limited cases, it's worth proxying the data through your servers (still in chunks of course).

When the experience demands immediate response. If users expect instant feedback based on file contents - like a profile photo appearing immediately with face detection, or a document preview generated during upload - the async nature breaks the UX. The round trip of upload → process → notify takes too long for truly interactive features.

Common Deep Dives

Here are some of the most common deep dives that come up when working with large blobs and how you can answer them.

"What if the upload fails at 99%?"

This question tests whether you understand chunked upload capabilities that all major cloud providers offer. When files exceed 100MB, you should use chunked uploads - S3 multipart uploads (5MB+ parts), GCS resumable uploads (any chunk size), or Azure block blobs (4MB+ blocks). The storage service tracks which parts uploaded successfully.

When a connection drops, the client doesn't start over. Instead, it queries which parts already uploaded; using ListParts in S3, checking the resumable session status in GCS, or listing committed blocks in Azure. If parts 1-19 succeeded but part 20 failed, the client resumes from part 20. This is especially critical for mobile users on flaky connections or large files that take hours to upload.

The implementation requires the client to track the upload session identifier. This is an upload ID in S3, a resumable upload URL in GCS, or block blob container URL in Azure. Some teams store this in localStorage so uploads can resume even after app restarts. Keep in mind that incomplete uploads cost money because cloud providers charge for stored parts, so set lifecycle policies to clean them up after 1-2 days.

"How do you prevent abuse?"

The most effective abuse prevention is simple - don't let users immediately access what they upload.

Implement a processing pipeline where uploads go into a quarantine bucket first. Run virus scans, content validation, and any other checks before moving files to the public bucket. This prevents someone from uploading malicious content and immediately sharing the link.

Set up automatic content analysis - image recognition to detect inappropriate content, file type validation to ensure a "photo" isn't actually an executable, size checks to prevent storage bombs. Only after these checks pass do you move the file to its final location and update the database status to "available."

This approach is much more robust than trying to detect abuse patterns in real-time. Even if someone bypasses your rate limiting and uploads malicious content, they can't use it until your systems approve it.

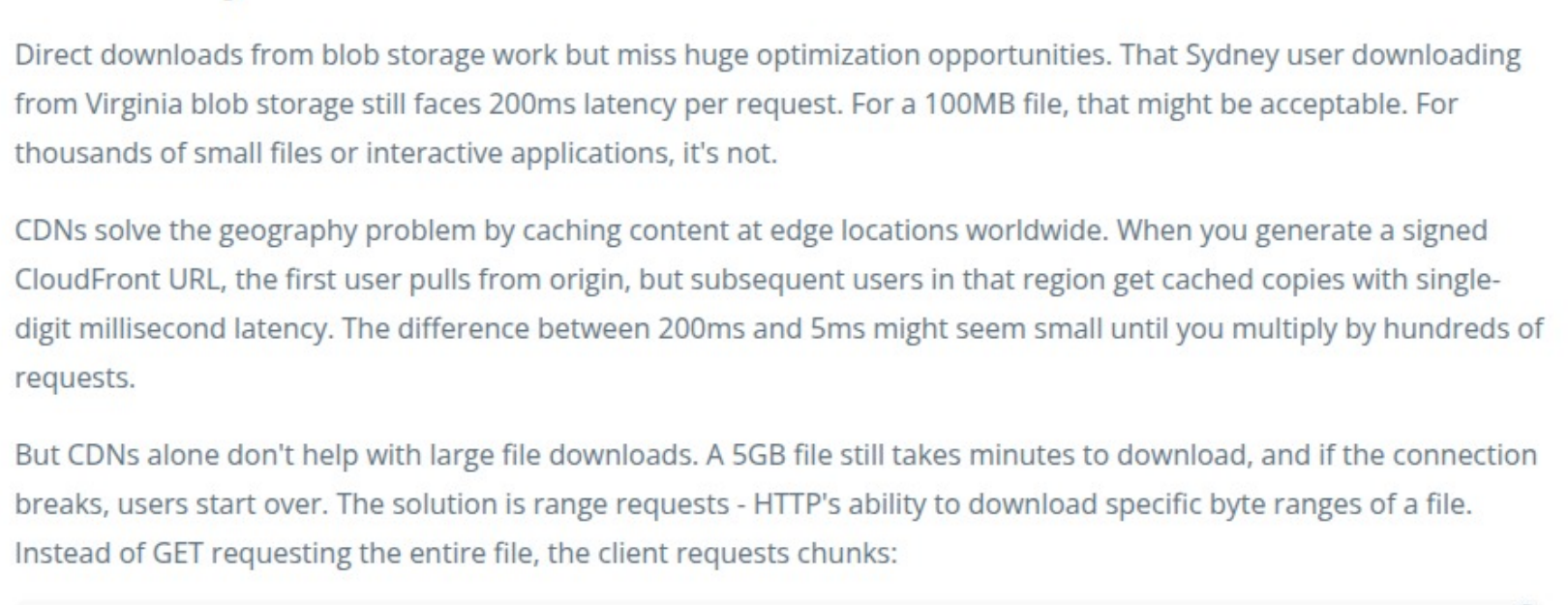
The processing delay also naturally throttles abuse - attackers can't immediately see if their upload worked, making automation harder.

Always include file size limits in the presigned URL conditions. Without these, someone could upload terabytes of data on a URL meant for a small image, exploding your storage costs.

"How do you handle metadata?"

Files almost always have associated metadata. Things like who uploaded it, what it's for, processing instructions, etc. But presigned URLs upload raw bytes to object storage. Interviewers want to know how you associate metadata with uploads.

As we discussed earlier, your metadata should exist in your primary database, separate from the file itself. The challenge is keeping them synchronized. When you generate a presigned URL, create the database record immediately with status 'pending'. This record includes the storage key you'll use for the upload. Now you have a reference before the file even exists.


Client
API Gateway
Server
Blob Storage - store file
Write Metadata
Event notification to update completion status
Primary DB - store metadata

Metadata

Some teams try to embed metadata in the storage object itself using tags or custom headers. S3 lets you attach up to 10 tags with 256 characters each. But this is limiting and makes queries painful. You can't efficiently find "all PDFs uploaded by user X in the last week" by scanning object tags. Keep rich metadata in your database where it belongs.

The storage key is your connection point. Use a consistent pattern like `uploads/{user_id}/{timestamp}/{uuid}` that includes useful info but prevents collisions. When storage events fire, they include this key, letting you find the exact database row to update. Never let clients specify their own keys - that's asking for overwrites and security issues.

For critical metadata that must be atomic with the upload, you can use conditional uploads. Generate presigned URLs that require specific metadata headers. If the client doesn't include them, the upload fails. But this adds complexity and most use cases don't need this level of strictness.

"How do you ensure downloads are fast?"

Direct downloads from blob storage work but miss huge optimization opportunities. That Sydney user downloading from Virginia blob storage still faces 200ms latency per request. For a 100MB file, that might be acceptable. For thousands of small files or interactive applications, it's not.

CDNs solve the geography problem by caching content at edge locations worldwide. When you generate a signed CloudFront URL, the first user pulls from origin, but subsequent users in that region get cached copies with single-digit millisecond latency. The difference between 200ms and 5ms might seem small until you multiply by hundreds of requests.

But CDNs alone don't help with large file downloads. A 5GB file still takes minutes to download, and if the connection breaks, users start over. The solution is range requests - HTTP's ability to download specific byte ranges of a file. Instead of GET requesting the entire file, the client requests chunks:

```
GET /large-file.zip
Range: bytes=0-10485759  (first 10MB)
```

This enables resumable downloads. Track which ranges completed and request only missing pieces after reconnection. Modern browsers and download managers handle this automatically if your storage and CDN support range requests (they all do). You just need to ensure your signed URLs don't restrict the HTTP verbs or headers needed.

For extreme cases like distributing massive datasets or game assets, some teams implement parallel chunk downloads. Split the file into parts, download 4-6 chunks simultaneously, then reassemble client-side. This can 3-4x download speeds by working around per-connection throttling. But the complexity is rarely worth it - most users are limited by their total bandwidth, not per-connection limits.

The pragmatic approach: serve everything through CDN with appropriate cache headers. Ensure range requests work for large files. Let the CDN and browser handle the optimization. Only consider exotic approaches like parallel downloads if you're distributing multi-gigabyte files and users complain about speeds despite good connectivity.

Conclusion

Large blob handling is a pattern that appears in almost every system design interview involving user-generated content. When you hear "video uploads," "file sharing," or "photo storage," immediately think about bypassing your servers for the actual data transfer. The key insight is shifting from moving bytes through your infrastructure to orchestrating access permissions and managing distributed state.

In interviews, demonstrate that you understand both the performance benefits and the complexity trade-offs. Show you know when to use direct uploads (anything over 100MB) and when not to (small files, compliance requirements). Lastly, don't forget the state synchronization challenges. This is where many candidates stumble.

Test Your Knowledge
Take a quick 15 question quiz to test what you've learned.
Start Quiz

Mark as read

Add a comment...
Comment
Anonymous
Posting as leetcode

Search comments
Sort By Old

Pritam Panda • 1 month ago
Hi Stefan,
How do you make sure that max limit on file size is forced ? For ex, like dropbox how do you make sure that file size more than 50 GB should be rejected immediately.
0 Reply

Ryan Lynch • 1 month ago • edited 1 month ago
Hey! This is discussed in the "Simple Direct Upload" section:
"Storage services let you add conditions that get validated during upload:
- content-length-range: Set min/max file sizes to prevent someone uploading 10GB when you expect 10MB"
3 Reply

Evan King • 1 month ago
Thanks Ryan!
0 Reply

VeryPeachCrow848 • 1 month ago • edited 1 month ago
Once multipart upload completes, the "chunks" no longer exist from the perspective of the storage service. They're all combined into a single object and it's as if you uploaded the entire file in one go.
In the video explanation 48:48 https://www.youtube.com/watch?v=UZ1ngy-kOI for Dropbox, Evan mentions that DropBox uses something called delta sync to find which chunks differ, and only those get updated. Is the chunking in that case self-managed by the service?
1 Reply

Evan King • 1 month ago
Both are true! S3's multipart upload API combines chunks into a single object after upload completes, but Dropbox implements its own chunking layer on top of that for delta sync. DB splits files into 4MB chunks, hashes each chunk, and keeps its own mapping of these hashes to S3 objects. When a file changes, Dropbox's client identifies which chunks changed through hash comparison, then only uploads those chunks to S3.
5 Reply

VeryPeachCrow848 • 1 month ago
Awesome thanks for clarifying, and thanks both of you for the great content!
0 Reply

CheerfulTurquoiseGuppy693 • 1 month ago • edited 1 month ago
The statement `If it's over 1MB and doesn't need SQL queries, it goes in blob storage.` is inconsistent with the quiz. In the quiz about when the upload via server becomes a problem, the expected answer is 100MB.
0 Reply

Evan King • 1 month ago
Yah let me update that, 1MB is small. 10 is a better rule of thumb (these are loose numbers as it very much depends on the set up)
1 Reply

Nimil Shah • 1 month ago
Hi Stefan,
Could you please explain how the presigned url is generated? In this blog it mention application's memory and no network calls are made to cloud storage. However in Design Dropbox, the presigned url is generated by making a call to S3 and then sent to client.
Thanks!
1 Reply

RainyCyanHippopotamus256 • 1 month ago • edited 1 month ago
We need to call S3 service to generate pre-signed urls. You can check boto3 "generate_presigned_url" API.
2 Reply

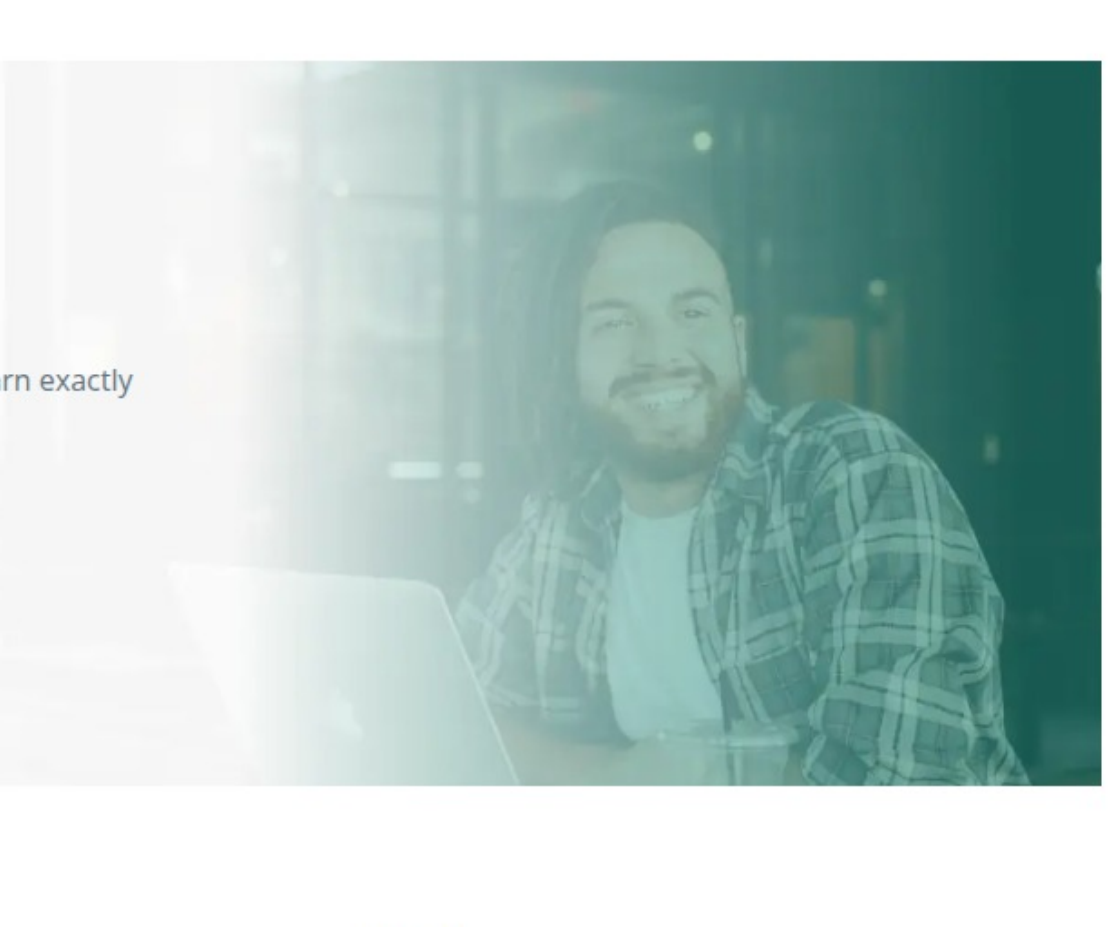
RainyCyanHippopotamus256 • 1 month ago • edited 1 month ago
Great content! However, I believe S3 or any other blob storage service doesn't provide event notifications at the chunk level for tracking progress. So, we might have to consider proxying the chunked data through our servers to track the progress accurately. To not allow chunked data through our servers, we can allow clients to call the API directly to update chunk progress from frontend, but it isn't safe and could lead to inconsistencies.
Edit: I just saw the video mentioning "Trust but Verify" approach where we allow the client to call the API to update the chunk progress, but verify if the chunk is actually uploaded in S3 before updating the DDB. Unfortunately, this doesn't work atleast with S3, since we don't have direct access to the chunked objects. We do have list parts API, but calling list parts API for every chunk looks a bit inefficient to me.
0 Reply

Evan King • 1 month ago
The client can track progress. If you need your server to know the status at any point, S3's ListParts API is efficient!
0 Reply

RainyCyanHippopotamus256 • 1 month ago • edited 1 month ago
I think we could have an API which would return the file upload status as an whole at chunk level. The API would call listparts API to return the chunks upload status. This could be polled at regular intervals on the client-side with adjustable polling interval.
However, calling listparts API for every chunk upload can cause issues if the TPS of listparts API is low. I don't know the exact permissible TPS of listparts API now, but I remember hitting TPS limits for one of the other AWS services in my work for determining the progress of ongoing jobs. This is something we should look probably look into if we were to implement it practically.
0 Reply

Show All Comments

Schedule a mock interview
Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.
Schedule A Mock Interview



Questions
Meta SWE Interview Questions
Amazon SWE Interview Questions
Google SWE Interview Questions
Engineering Manager (EM) Interview Questions

Learn
Learn System Design
Learn DSA
Learn Behavioral
Learn ML System Design
Guided Practice

Links
FAQ
Pricing
Gift Mock Interviews
Gift Premium
Become a Coach
Our Coaches
Employers
Hello Interview Premium

Legal
Terms and Conditions
Privacy Policy

Contact
About Us
support@hellointerview.com
7511 Greenwood Ave North
Unit #4238 Seattle
WA 98103

© 2025 Optick Labs Inc. All rights reserved.